

Purpose

This project aims at helping you **think like a systems engineer**, not just use tools.

You will study a real-world data system by analyzing its codebase and answer:

- What problem does this system solve?
 - What design decisions were made?
 - What trade offs do those decisions introduce?
 - How does the system behave under stress?
-

What This Project Is (and Is Not)

This project IS:

- Reverse engineering a system
- Connecting **code** → **concepts** → **system design**
- Demonstrating understanding through **experiments**

This project is NOT:

- A summary of documentation
 - A walkthrough of files/classes
 - A tutorial-style explanation
-

Core Expectations

Every team must demonstrate:

1. Execution Understanding

You must trace **one complete execution path** in the system.

Examples:

- Write path (data ingestion → storage)
- Query path (request → execution → result)
- Message flow (producer → broker → consumer)

You must reference **actual code (files/functions)**

2. Design Decisions

Identify at least **three key design decisions**.

For each:

- Where is it implemented in code?
 - What problem does it solve?
 - What tradeoff does it introduce?
-

3. Concept Mapping

Relate your system to concepts taught in class:

- Storage (B-tree / LSM / columnar)
- Execution (MapReduce / DAG)
- Streaming / ingestion
- Partitioning / replication
- Reliability / fault tolerance

Minimum: 4 concepts

4. Experiment / Modification (Mandatory)

You must:

- Modify or isolate a part of the system
- Observe behavior
- Explain results

Examples:

- Change parameters (batch size, thresholds)
 - Simulate skew or failure
 - Compare performance under different inputs
-

5. Failure Analysis

You must answer at least TWO:

- What happens when data size increases significantly?
 - What happens under skew?
 - What happens if a component fails?
 - What assumptions does this system rely on?
-

Presentation Expectations

Each team will present:

Structure (Strict)

- 10 min → System overview
 - 10 min → Deep dive (one mechanism)
 - 10 min → Experiment/demo
 - 5 min → Key insights
-

Viva

You should be able to answer:

- Why was this design chosen?
 - What alternatives exist?
 - What breaks under real-world conditions?
 - How would you improve the system?
-

Evaluation Criteria

Component	Weight
System understanding	25%
Concept mapping	20%
Experiment	25%
Presentation	20%
Defense	10%

What Will Be Penalized

- No code references
 - Only diagrams / high-level explanation
 - No experiment
 - Copying documentation
 - Vague claims without evidence
-

Suggested Approach

1. Identify entry point (API/function)
 2. Trace one path end-to-end
 3. Understand surrounding components
 4. Map to concepts
 5. Design and run experiment
-

Key Principle

If you cannot point to code, you have not understood the system.

Deliverables

- Report (markdown file)
 - Code / experiment artifacts (pushed on github)
 - Presentation slides
-

Suggested timeline

- Week 1 → Understand system + trace execution
 - Week 2 → Deep dive + concept mapping
 - Week 3 → Experiment + analysis
 - Week 4 → Final report + presentation
-